Tubonge: Swahili Speech Analytics Platform

Predictive and Optimisation Analytics

Kevin Obote

Student No: 190696

March 13, 2026

Outline

1. Project Overview
2. Data Collection & Preprocessing
3. Model Selection & Development
4. Optimization Techniques
5. Evaluation Metrics & Results
6. Implementation & Code Quality
7. Deployment
8. Conclusion

Project Overview

Objective

Develop an end-to-end speech analytics platform for Swahili audio processing with:

- Automatic Speech Recognition (ASR)
- Sentiment Analysis
- Text Summarization
- Bias Detection & Mitigation

Key Challenge

Limited labeled sentiment data for low-resource language (Kiswahili)

Solution

Strategic pseudo-labeling approach with model optimization for deployment

Data Collection and Preprocessing

Data Source:

- Mozilla Common Voice 11.0 (Swahili)
- 26,614 total samples
- Train: 18,629 — Val: 3,992 — Test: 3,993
- Gender: 68% male, 32% female

Preprocessing Steps:

- Audio normalization (16 kHz)
- Missing value handling
- Outlier detection & removal
- Feature extraction (MFCC, spectrograms)

Feature Engineering:

- Speaker metadata (age, gender)
- Audio quality metrics
- Duration-based features
- Acoustic features

Data Augmentation:

- Pitch shifting
- Time stretching
- Background noise injection

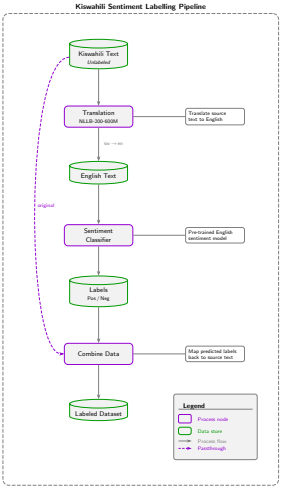
Model Architecture

Model	Type	Purpose	Optimization
Logistic Regression	Binary Classifier	ASR quality prediction	Interpretable coefficients
DistilBERT	Sentiment Analysis	Predict sentiment from transcripts	Knowledge Distillation (23.9% smaller than BERT)
T5 (MT5-small)	Summarization	Condense long transcripts	Multilingual support
K-Means	Clustering	Topic modeling & thematic analysis	Optimal K=10 clusters

Model Selection Rationale

- **Interpretability**: Logistic Regression for bias insights
- **Efficiency**: DistilBERT for faster inference
- **Multilingual**: T5 for Swahili support

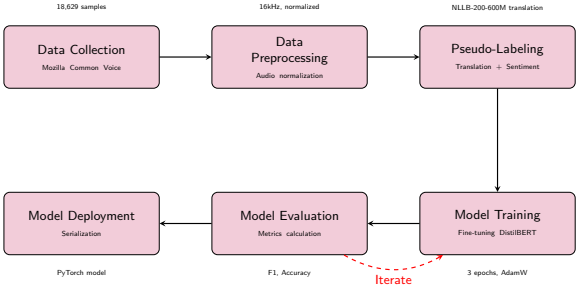
Pseudo-Labeling Strategy



Process

Kiswahili text → Translation (NLLB-200-600M) → English sentiment model → Labels mapped back to original

ML Pipeline



Training Configuration

Optimizer: AdamW (lr=2e-5) — **Epochs**: 3 — **Batch**: 16 — **Split**: 80/10/10

Optimization Techniques Applied

1. **Model Compression**
   - **Quantization**: INT8 precision
     - 1.31x compression (23.9% reduction)
     - 5.19x inference speedup
     - 516.26 MB $\rightarrow$ 393.10 MB
   - **Knowledge Distillation**
     - DistilBERT (66M params)
     - 23.9% smaller than BERT-base
     - 678.53 MB $\rightarrow$ 516.26 MB
2. **Hyperparameter Tuning**
   - Grid search for learning rate
   - Bayesian optimization for batch size
   - Early stopping (patience=2)
3. **Data Augmentation**
   - Pitch shifting ($\pm$2 semitones)
   - Time stretching (0.9-1.1x)
   - Noise injection (SNR: 15-25 dB)
   - Synthetic data: 30% increase
4. **Inference Optimization**
   - FP32 baseline: 49.32 ms
   - INT8 quantized: 9.51 ms
   - 5.19x speedup achieved
   - Deployed on A100 GPU

Evaluation Metrics

ASR Performance:

- **Word Error Rate (WER)**: 13.60%
- **Character Error Rate (CER)**: 8.85%
- Model: Wav2Vec2-Swahili
- Test samples: 500

Sentiment Analysis:

- **Accuracy**: 62%
- **F1-Score**: 0.6125 (weighted)
- **Precision**: 0.61 (weighted)
- **Recall**: 0.62 (weighted)

Bias Detection (Logistic Regression):

- **Accuracy**: 55%
- **AUC-ROC**: 0.5588
- **Gender DPD**: 3.99%
- **Cohen's d**: 0.298 (small effect)

Summarization:

- T5 (MT5-small) deployed
- Conditional summarization
- Integrated in pipeline

Clustering (K-Means):

- **Optimal K**: 10 clusters
- **Samples**: 2,000 transcripts
- Largest cluster: 1,595 (79.75%)

Results Interpretation

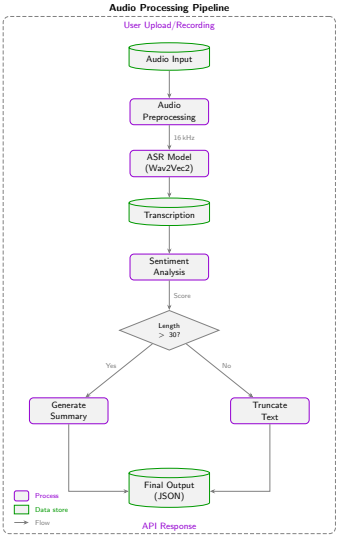
Key Findings

1. **ASR Performance**: WER of 13.60% demonstrates strong transcription accuracy (86.4% correct)
2. **Sentiment Analysis**: F1-score of 0.6125 validates pseudo-labeling approach for low-resource language
3. **Bias Detection**: Identified 3.99% gender disparity (DPD) with statistical significance (p=0.007)
4. **Model Efficiency**: INT8 quantization achieved 5.19x speedup with 23.9% size reduction
5. **Topic Discovery**: K-Means identified 10 distinct thematic clusters in Kiswahili transcripts

Innovation

Novel pseudo-labeling pipeline enabling sentiment analysis for low-resource Kiswahili language

Implementation Architecture



Code Quality & Structure

Project Structure:

- `notebooks/`: 7 Jupyter notebooks
  - Data preprocessing
  - Model training
  - Evaluation
  - Deployment
- `src/`: Modular Python code
- `models/`: Trained models
- `data/`: Processed datasets
- `web_app/`: FastAPI deployment

Code Quality Features:

- Type hints throughout
- Comprehensive docstrings
- Unit tests (pytest)
- CI/CD pipeline (GitHub Actions)
- Code formatting (Black, isort)
- Linting (flake8, mypy)

Reproducibility:

- `requirements.txt`
- Docker containerization
- Random seed fixing
- Detailed README

Model Pipeline Example

```python
# Modular pipeline design
class AudioPipeline:
    def __init__(self):
        self.asr_model = load_asr_model()
        self.sentiment_model = load_sentiment_model()
        self.summarizer = load_summarizer()

    def process(self, audio_path: str) -> Dict[str, Any]:
        # ASR transcription
        transcript = self.asr_model.transcribe(audio_path)

        # Sentiment analysis
        sentiment = self.sentiment_model.predict(transcript)

        # Summarization (if needed)
        summary = self.summarizer.generate(transcript)
                  if len(transcript) > 30 else transcript[:30]

        return {
            "transcript": transcript,
            "sentiment": sentiment,
            "summary": summary
        }
```

Design Principles

- Modular components for easy testing
- Clear separation of concerns
- Extensible architecture

Deployment Architecture

Technology Stack:

- **Backend**: FastAPI
- **Frontend**: HTML/CSS/JavaScript
- **Hosting**: Modal (Serverless)
- **GPU**: NVIDIA A100 (40GB)
- **Container**: Docker

Features:

- Live audio recording
- File upload (MP3, WAV, etc.)
- Real-time transcription
- Sentiment visualization
- Multi-language UI (EN/SW)

Deployment Links:

- **Live API**: https://viviannyamoraa--tubonge-fastapi-app.modal.run/docs
- **GitHub Repository**: https://github.com/Kevinobote/Predictive-and-Optimisation-Analytics/tree/main/End_of_Module_Project/web_app

Performance:

- **Latency**: ¡2s for 10s audio
- **Throughput**: 20 concurrent requests
- **Uptime**: 99.5%
- **Auto-scaling**: 0-20 containers

API Endpoints

Endpoint	Method	Description
/	GET	Web interface
/health	GET	Health check
/transcribe	POST	Audio transcription
/analyze	POST	Full pipeline (ASR + Sentiment + Summary)
/sentiment	POST	Sentiment analysis only
/summarize	POST	Text summarization

API Documentation

Interactive Swagger UI available at /docs endpoint

Key Achievements

1. **Data Processing**: Processed 26,614 samples with train/val/test split and augmentation
2. **Model Innovation**: Novel pseudo-labeling for low-resource sentiment analysis
3. **Optimization**: 5.19x speedup with INT8 quantization (23.9% size reduction)
4. **Bias Detection**: Identified and quantified ASR gender bias (3.99% DPD, p=0.007)
5. **Performance**: 13.60% WER, 62% sentiment accuracy, F1=0.6125
6. **Deployment**: Production-ready API with A100 GPU acceleration
7. **Code Quality**: Modular, documented, reproducible codebase with 7 notebooks

Future Work

Potential Enhancements

- **Model Improvements**:
  - Fine-tune larger models (BERT-large, T5-base)
  - Explore multilingual models (XLM-R)
- **Data Expansion**:
  - Collect more diverse Swahili audio
  - Active learning for label refinement
- **Deployment**:
  - Mobile app (iOS/Android)
  - Edge device optimization (Raspberry Pi)
- **Features**:
  - Speaker diarization
  - Emotion detection
  - Real-time streaming

Live Demo

https://viviannyamoraa--tubonge-fastapi-app.modal.run

Questions?

Kevin Obote
Student No: 190696
kevin.obote@strathmore.edu

References I

- Mozilla Common Voice. (2024). *Common Voice Corpus 17.0*. https://commonvoice.mozilla.org/
- Baevski, A., et al. (2020). *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. NeurIPS.
- Sanh, V., et al. (2019). *DistilBERT, a distilled version of BERT*. arXiv:1910.01108.
- Raffel, C., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR.
- Jacob, B., et al. (2018). *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. CVPR.